Audit-Bericht: CrewBase Arbeitszeitverwaltung

Datum: 02. März 2026 **Auditor:** Manus AI (Full-Stack-Entwickler, QA-Auditor, DSGVO-Experte)

---

1. Zusammenfassung & Management Summary

Die Architektur der CrewBase Arbeitszeitverwaltungs-App ist insgesamt **solide, logisch strukturiert und performant**. Die Kernfunktionen wie Zeiterfassung, Lohnberechnung und Dienstplanung sind funktional implementiert. Das System erfüllt die wesentlichen Anforderungen an eine moderne, webbasierte Verwaltungslösung.

Das Audit identifiziert jedoch **zwei kritische Sicherheitslücken (DSGVO)** und mehrere **mittelschwere Risiken** in den Bereichen Datenschutz und Datenintegrität. Zudem gibt es Optimierungspotenzial bei der User Experience (UX) und der Automatisierung.

Bereich	Bewertung	Kritikalität
Kern-Logik & Rechenwege	✅ **Sehr Gut**	Gering
Dashboard-Spezifikationen	✅ **Gut**	Gering
DSGVO & Datenschutz	❌ **Kritisch**	**Hoch**
Design & Effizienz	🟠 **Mittel**	Mittel

Handlungsempfehlung: Die identifizierten **DSGVO-Lücken müssen umgehend geschlossen werden**. Die weiteren Empfehlungen zur Effizienz und zum Design sollten zur Steigerung der Nutzerakzeptanz und zur Reduzierung von Fehlbedienungen mittelfristig umgesetzt werden.

---

2. Kern-Logik & Rechenwege (Audit-Fokus)

Die mathematische und logische Korrektheit der Berechnungen ist **gewährleistet**. Das Modul `utils/lohnberechnung.py` ist das Herzstück und wurde detailliert geprüft.

Bewertung: ✅ Sehr Gut

Feature	Status	Details & Verifizierung
Arbeitszeitkonto	✅ **Korrekt**	Die Berechnung von Plus-/Minusstunden durch `ist_stunden - soll_stunden` ist korrekt implementiert und wird in der `zeitauswertung.py` sauber dargestellt. Die Soll-Stunden werden dynamisch aus dem Dienstplan oder den Stammdaten bezogen.
Zuschlagslogik	✅ **Korrekt**	Die Zuschlags-Matrix in `lohnberechnung.py` wendet Sonntagszuschläge (+50%) und Feiertagszuschläge (+100%) korrekt an. Die Prüfung der individuellen Mitarbeiter-Flags (`sonntagszuschlag_aktiv` , `feiertagszuschlag_aktiv`) ist implementiert. Die Regel "Feiertag auf Sonntag = 100%" wird korrekt umgesetzt.
Urlaubsberechnung	✅ **Korrekt**	Der Abzug von genehmigten Urlaubstagen vom Jahreskontingent ist logisch korrekt. Die Darstellung im Jahreskalender und der PDF-Export funktionieren wie spezifiziert.
Lohn-Schnittstelle	✅ **Korrekt**	Der DATEV-Export (`utils/datev_export.py`) generiert eine CSV-Datei mit den korrekten Lohnarten-Schlüsseln (1000, 1100, 1200) und dem geforderten Format (Semikolon-Trenner, Komma-Dezimal).
Pausenlogik	✅ **Korrekt**	Die automatische Pausenberechnung nach § 4 ArbZG (> 6h = 30min, > 9h = 45min) ist in `lohnberechnung.py` korrekt implementiert.

---

3. Spezifikationen der Dashboards (Struktur-Check)

Die Dashboards sind logisch aufgebaut und die geforderten Funktionen sind vorhanden.

Bewertung: ✅ Gut

Feature	Status	Details & Verifizierung
Mitarbeiter-Dashboard	✅ **Erfüllt**	Alle geforderten Sektionen (Stammdaten, Dienstplan, Lohn, Plauderecke) sind vorhanden. Die Änderungsantrags-Funktion mit Begründung ist implementiert.
Sonderberechtigung Außendienst	✅ **Erfüllt**	Die Prüfung in `utils/device_management.py` und `mitarbeiter_dashboard.py` stellt sicher, dass nur Mitarbeiter mit dem Flag `mobile_zeiterfassung` die Stempeluhr im eigenen Dashboard sehen. Die Namen (Nadine Lutschin, Melanie Lasinski, Hans Jürgen Lasinski) sind zwar nicht hartcodiert, aber die Logik über das Admin-setzbare Flag ist flexibler und korrekt.
Admin-Dashboard	✅ **Erfüllt**	Alle Verwaltungsfunktionen (Doku-Upload, Dienstplan-Vorlagen, Korrekturmodus, Export) sind implementiert. Der Korrekturmodus in `admin_dashboard.py` (Zeile 1091) erzwingt eine Begründung.
Sonderwunsch: Fernando (Lila)	❌ **Nicht erfüllt**	Es wurde **keine** spezifische Implementierung gefunden, die den Urlaub von Fernando im Kalender lila (`#8A2BE2` , `purple` , `violet`) darstellt. Die Kalenderfarben sind für alle Mitarbeiter gleich (Gelb für Urlaub). Dies ist eine kleine Abweichung von der Spezifikation.

---

4. DSGVO, Datenschutz & Security-Audit

In diesem Bereich wurden **kritische Mängel** identifiziert, die ein **hohes Risiko** für die Datensicherheit und DSGVO-Konformität darstellen.

Bewertung: ❌ Kritisch

Kriterium	Status	Risiko & Empfehlung
Rollen- & Rechtekonzept	❌ **Kritisch**	**Problem:** Die Supabase Row Level Security (RLS) Policies sind **unzureichend**. Die `*_admin_all` Policies (`CREATE POLICY users_admin_all ON users FOR ALL USING (true);`) gewähren dem Admin zwar vollen Zugriff, aber es fehlen die entscheidenden Policies für Mitarbeiter. **Mitarbeiter können aktuell die Daten anderer Mitarbeiter lesen!** Zwar filtert die App-Logik die Daten (z.B. `get_mitarbeiter_by_user_id`), aber ein technisch versierter Nutzer könnte die API direkt abfragen. **Empfehlung: UMGEHEND** RLS-Policies für Mitarbeiter implementieren, die den Zugriff strikt auf die eigene `user_id` beschränken. Beispiel: `CREATE POLICY mitarbeiter_user_select ON mitarbeiter FOR SELECT USING (auth.uid() = user_id);`
Datensparsamkeit & Verschlüsselung	❌ **Kritisch**	**Problem:** Arbeitsverträge und Gesundheitsausweise werden in Supabase Storage **unverschlüsselt** abgelegt (`upload_file_to_storage`). Der Zugriff darauf ist nur durch die RLS der `mitarbeiter` -Tabelle (die den Pfad enthält) geschützt. Fällt die RLS, sind die Dokumente offen. **Empfehlung: UMGEHEND** eine client- oder serverseitige Verschlüsselung für sensible Dokumente implementieren, bevor sie in den Storage geladen werden. Die Schlüssel müssen sicher verwaltet werden.
Löschfristen	🟠 **Mittel**	**Problem:** Es existiert **keine** automatisierte Logik zur Löschung von Mitarbeiterdaten nach Austritt oder Ablauf gesetzlicher Aufbewahrungsfristen (z.B. 10 Jahre für Lohnunterlagen). Daten bleiben unbegrenzt im System. **Empfehlung:** Ein Cron-Job oder eine serverseitige Funktion implementieren, die regelmäßig ausgetretene Mitarbeiter prüft und deren Daten nach Ablauf der Fristen anonymisiert oder löscht.
Protokollierung (Korrektur)	✅ **Gut**	**Problem:** Die manuelle Korrektur von Zeitbuchungen durch Admins wird in der `zeiterfassung` -Tabelle mit `korrektur_grund` und `korrigiert_von_admin` geloggt. Dies ist gut, aber nicht manipulationssicher. Ein Admin könnte den Log-Eintrag in der Datenbank direkt ändern.

Kriterium	Status	Risiko & Empfehlung
		Empfehlung: Für eine höhere Revisionssicherheit sollte eine separate, unveränderliche `audit_log`-Tabelle eingeführt werden, die nur `INSERT`-Rechte hat.
Plauderecke	✅ **Gut**	**Problem:** Der Chat ist für alle Mitarbeiter des Betriebs sichtbar. Dies ist datenschutzrechtlich unproblematisch, solange keine sensiblen Daten geteilt werden. **Empfehlung:** Ein expliziter Hinweis im UI der Plauderecke (`st.info("Dieser Chat ist für alle Mitarbeiter sichtbar. Bitte teilen Sie hier keine privaten oder sensiblen Informationen.")`) würde die Transparenz erhöhen.

---

5. Design & Effizienz

Das Design ist funktional, aber es gibt deutliches Potenzial zur Verbesserung der User Experience und zur Reduzierung von Redundanzen.

Bewertung: 🟠 Mittel

Kriterium	Status	Empfehlung
Stil & Lesbarkeit	🟠 **Mittel**	Das dunkle Design ist ein guter Anfang, aber die Kontraste und Schriftgrößen sind nicht durchgängig optimiert. Die Verwendung von `st.metric` ist gut, aber die Gesamtanmutung wirkt noch sehr technisch. Ein "edler Style" wird nicht erreicht. **Empfehlung:** Ein professionelles UI-Kit oder eine einheitliche CSS-Design-Systematik implementieren. Abstände, Schriftarten und Farben vereinheitlichen.
Redundanz-Check	🟠 **Mittel**	**Problem:** Im Admin-Dashboard gibt es die Tabs "Zeiterfassung", "Zeitauswertung / Lohn" und "Lohnabrechnung". Diese überlappen sich stark in ihrer Funktion. **Empfehlung:** Die drei Tabs zu einem einzigen Tab **"Lohn & Zeiten"** zusammenfassen. Dieser Tab könnte per Sub-Navigation (z.B. `st.selectbox`) zwischen Mitarbeiter-Auswertung, Korrektur und DATEV-Export umschalten.
Automatisierung (E-Mail)	🟠 **Mittel**	**Problem:** Es gibt E-Mail-Trigger für Urlaubsgenehmigungen, aber nicht für andere wichtige Ereignisse. **Empfehlung:** Zusätzliche E-Mail-Benachrichtigungen implementieren für: 1) Admin bei neuem Änderungsantrag in Stammdaten, 2) Mitarbeiter bei neuem Dienstplan, 3) Mitarbeiter bei neuer Nachricht in der Plauderecke (optional, per User-Einstellung).

Ende des Berichts.